

DESENVOLVIMENTO DE UM MOTOR ANALÍTICO E SIMULADOR MATEMÁTICO DE COMPORTAMENTO DE MEMÓRIA VIRTUAL PARA DISPOSITIVOS DE BORDA

Disciplina: DCC403

Universidade Federal de Roraima – UFRR



José Carvalho Neto



Junho/2026



? PROBLEMA

Dispositivos de borda possuem recursos limitados:

- Pouca memória RAM
- Pouco armazenamento
- Baixo consumo energético

Quando a memória é insuficiente para atender às demandas dos processos:

- Ocorrem Page Faults (falhas de página)
- Aumentam as operações de disco
- Surge o fenômeno de **Hiperpaginação (Thrashing)**

🎯 OBJETIVO

Desenvolver um simulador capaz de analisar o comportamento da memória virtual em dispositivos de borda, permitindo avaliar diferentes políticas de substituição de páginas e seus impactos no desempenho do sistema.

DISPOSITIVOS DE BORDA



Gateways



Câmeras IP



Placas Embarcadas



Sensores Inteligentes

Consequência

Memória insuficiente



Mais Page Faults



Menor Desempenho



Thrashing (Hiperpaginação)



IMPORTANTE

Entender o comportamento da memória virtual é essencial para projetar sistemas eficientes, confiáveis e com baixo consumo de recursos em ambientes de borda.



OBJETIVO GERAL

Desenvolver um simulador de memória virtual utilizando paginação sob demanda, capaz de analisar o comportamento de diferentes políticas de substituição de páginas em dispositivos de borda.



OBJETIVOS ESPECÍFICOS

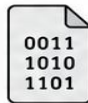
- ✓ Processar traços de memória contendo acessos de leitura e escrita.
- ✓ Implementar o algoritmo FIFO (First In First Out).
- ✓ Implementar o algoritmo LRU (Least Recently Used).
- ✓ Implementar o algoritmo Random.
- ✓ Medir e contabilizar o número de Page Faults.
- ✓ Medir e contabilizar o número de Escritas em Disco.
- ✓ Calcular a taxa de Page Faults.
- ✓ Estudar o fenômeno de Hiperpaginação (Thrashing).
- ✓ Investigar a Anomalia de Bélády utilizando a política FIFO.



IMPORTÂNCIA

A compreensão do comportamento da memória virtual permite projetar sistemas mais eficientes, reduzir o número de falhas de página, evitar a hiperpaginação e melhorar o desempenho de aplicações executadas em ambientes com recursos limitados.

O QUE SERÁ ANALISADO



Entrada: Trace de Memória

Arquivo contendo endereços de memória e operações (R/W).



Memória Física (Quadros)

Conjunto limitado de quadros que armazena as páginas carregadas.



Page Fault

Quando a página não está na memória física, ocorre uma falha de página.



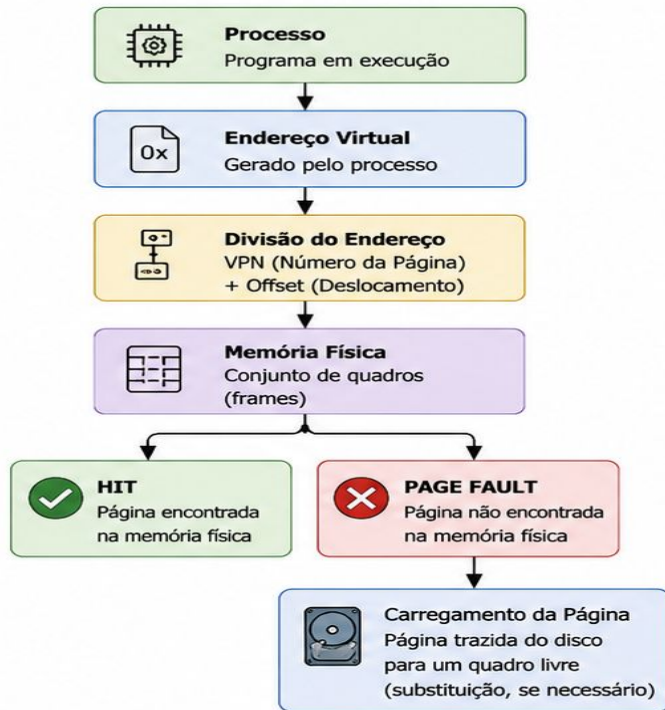
Escrita em Disco

Se a página removida foi modificada (dirty), ela é escrita no disco.



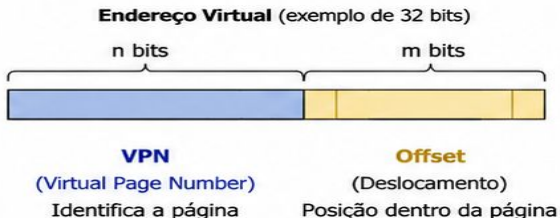
A memória virtual utiliza paginação sob demanda para gerenciar os acessos à memória.

FLUXO DE ACESSO



DIVISÃO DO ENDEREÇO VIRTUAL

O endereço virtual é dividido em:
VPN + Offset



Tamanho da página = 2^m bytes
Número de páginas possíveis = 2^n
Endereço Virtual = VPN + Offset



COMO FUNCIONA

- O processo gera um endereço virtual.
- Esse endereço é dividido em VPN e Offset.
- O sistema verifica se a página (VPN) está presente na memória física.
- Se estiver → **HIT** (acesso rápido).
- Se não estiver → **PAGE FAULT**.
- A página é carregada do disco para um quadro livre (ou substituindo outra página, conforme o algoritmo).



OBSERVAÇÕES

- A paginação permite que processos usem um espaço de memória maior do que a memória física disponível.
- O desempenho depende fortemente da taxa de Page Faults.
- Em dispositivos de borda, minimizar Page Faults é essencial devido à limitação de memória e ao alto custo de acesso ao armazenamento.



O simulador processa um trace de memória e, para cada acesso, aplica a política de substituição escolhida.

1 ENTRADA (TRACE)

Arquivo de texto contendo acessos à memória no formato:

```
001fa3c0 R
7f32b1a0 W
0040a120 R
...
```

→ Endereço hexadecimal

→ Operação (R/W)

i Parâmetros informados pelo usuário

- Algoritmo: FIFO, LRU ou Random
 - Tamanho da página (bytes)
 - Tamanho da memória física (bytes)
- Define o número de quadros disponíveis

2 PROCESSAMENTO



Leitura do Trace

Lê cada linha do arquivo sequencialmente.



Conversão e Cálculo

Converte o endereço hexadecimal para decimal e calcula o VPN (número da página).



Verificação na Memória Física

Verifica se a página está presente em algum dos quadros.



HIT

Página encontrada (acesso rápido)



PAGE FAULT

Página não encontrada (falha de página)



Substituição de Página

Aplica a política escolhida (FIFO, LRU ou Random) para selecionar uma vítima, se necessário.



Escrita em Disco (se necessário)

Se a página removida estava modificada (dirty), ela é escrita no disco (contabilizada).

3 SAÍDA (ESTATÍSTICAS)

Ao final da execução, o simulador exibe:



Total de Acessos

Quantidade total de referências processadas.



Page Faults

Número total de falhas de página ocorridas.



Escritas em Disco

Número de páginas sujas (dirty) gravadas no disco.



Taxa de Falhas (%)

Percentual de Page Faults em relação ao total de acessos.

i Observação

As estatísticas permitem comparar o desempenho das diferentes políticas de substituição sob as mesmas condições de execução.

RESUMO DO FUNCIONAMENTO



Lê o Trace



Calcula VPN



Verifica na memória



Hit (sucesso)



Page Fault (falha)



Substitui página (se necessário)



Escreve no disco (se suja)



Atualiza estatísticas



O simulador implementa três políticas de substituição de páginas: FIFO, LRU e Random.

FIFO (First In First Out)



Remove da memória a página que está há mais tempo na fila (a mais antiga).

COMO FUNCIONA

- Cada página recebe uma ordem de chegada.
- Quando ocorre Page Fault e não há quadros livres, remove-se a página mais antiga.

EXEMPLO

Referência de páginas: 1 2 3 4 1 2 5
Quadros disponíveis: 3

	1	2	3	4	1	2	5
Quadro 1	1	1	1	4	1	4	5
Quadro 2	-	2	2	2	1	1	1
Quadro 3	-	-	3	3	3	2	2

 Página carregada
 Página substituída
 Quadro livre

LRU (Least Recently Used)



Remove da memória a página que não é utilizada há mais tempo (a menos recente).

COMO FUNCIONA

- Cada acesso atualiza o "último uso" da página.
- Quando ocorre Page Fault, remove-se a página com o menor tempo de último uso.

EXEMPLO

Referência de páginas: 1 2 3 4 1 2 5
Quadros disponíveis: 3

	1	2	3	4	1	2	5
Quadro 1	1	1	1	4	4	4	5
Quadro 2	-	2	2	2	2	1	2
Quadro 3	-	-	3	3	1	1	1

 Página carregada
 Página substituída
 Quadro livre

RANDOM (Substituição Aleatória)



Remove da memória uma página escolhida de forma aleatória.

COMO FUNCIONA

- Quando ocorre Page Fault e não há quadros livres, escolhe-se aleatoriamente uma página para ser removida.

EXEMPLO

Referência de páginas: 1 2 3 4 1 2 5
Quadros disponíveis: 3
(Uma possível execução)

	1	2	3	4	1	2	5
Quadro 1	1	1	1	4	4	4	4
Quadro 2	-	2	2	2	2	2	5
Quadro 3	-	-	3	3	1	1	1

 Página carregada
 Página substituída
 Quadro livre



OBSERVAÇÃO

Cada algoritmo possui características diferentes e pode apresentar desempenhos distintos dependendo do padrão de referência de memória e da quantidade de quadros disponíveis.

Critérios de avaliação utilizados:

- ✓ Número de Page Faults
- ✓ Número de Escritas em Disco
- ✓ Taxa de Falhas (%)



7 ESTRUTURA DE DADOS

Cada quadro da memória física é representado pela estrutura Frame.

ESTRUTURA FRAME

```
typedef struct {  
    unsigned long page;           // Número da página  
    int valid;                    // Bit de validade (0 ou 1)  
    int dirty;                    // Dirty Bit (0 ou 1)  
    unsigned long last_used;      // Último instante de uso (para LRU)  
    unsigned long arrival_time;  // Instante de chegada (para FIFO)  
} Frame;
```

MEMÓRIA FÍSICA (EXEMPLO)

Quadros disponíveis: 4

Índice	page	valid	dirty	last_used	arrival_time
0	15	1	1	105	10
1	23	1	0	110	8
2	-	0	0	-	-
3	42	1	1	115	12

 Quadro ocupado

 Página suja (dirty = 1)

 Quadro livre (valid = 0)

SIGNIFICADO DOS CAMPOS

123

page

Armazena o número da página virtual que está carregada no quadro.



valid

Indica se o quadro está válido.
1 = ocupado | 0 = livre



dirty

Indica se a página foi modificada.
1 = página suja (deve ser escrita no disco) | 0 = limpa



last_used

Armazena o instante do último acesso à página.
Usado pelo algoritmo LRU.



arrival_time

Armazena o instante em que a página entrou no quadro.
Usado pelo algoritmo FIFO.

COMO É UTILIZADA

- ✓ O simulador mantém um vetor (array) de Frames representando os quadros.
- ✓ Para cada acesso ao trace, a estrutura é consultada e atualizada.
- ✓ Os campos last_used e arrival_time permitem que as políticas LRU e FIFO escolham a vítima corretamente.
- ✓ O campo dirty garante que páginas modificadas sejam escritas no disco antes de serem removidas.



IMPORTANTE

A correta manutenção dessas informações é essencial para garantir a precisão das estatísticas (Page Faults e Escritas em Disco) e o comportamento adequado de cada algoritmo.

VANTAGENS DESSA REPRESENTAÇÃO

- Estrutura simples e eficiente.
- Permite fácil implementação dos algoritmos.
- Suporte completo às métricas exigidas.



Os testes foram realizados com o trace de exemplo utilizando 4 quadros de memória física.

CONFIGURAÇÃO DOS TESTES



- Trace de entrada: trace_exemplo.txt
- Tamanho da página: 4096 bytes (4 KB)
- Memória física: 4 quadros
- Algoritmos testados: FIFO, LRU e Random

TRACE UTILIZADO



Passo	Endereço (hex)	Operação	VPN
1	001fa3c0	R	1
2	7f32b1a0	W	7
3	0040a120	R	4
4	001fa3c0	R	1
5	7f32b1a0	W	7
6	0040a120	R	4
7	001fa3c0	R	1
8	7f32b1a0	W	7
9	0040a120	R	4
10	001fa3c0	R	1

Os endereços foram convertidos para VPN (número da página).

COMPARAÇÃO DOS ALGORITMOS (4 QUADROS)

Passo	VPN	FIFO		LRU		Random	
		Quadros após referência	PF	Quadros após referência	PF	Quadros após referência	PF
1	1	1 - - -	F	1 - - -	F	1 - - -	F
2	7	1 7 - -	F	1 7 - -	F	1 7 - -	F
3	4	1 7 4 -	F	1 7 4 -	F	1 7 4 -	F
4	1	1 7 4 -	H	1 7 4 -	H	1 7 4 -	H
5	7	1 7 4 -	H	1 7 4 -	H	1 7 4 -	H
6	4	1 7 4 -	H	1 7 4 -	H	1 7 4 -	H
7	1	1 7 4 -	H	1 7 4 -	H	1 7 4 -	H
8	7	7 7 4 1	F	7 4 1 7	F	7 4 1 7	F
9	4	7 4 4 1	H	7 4 1 7	H	7 4 1 7	H
10	1	7 4 1 1	H	7 4 1 7	H	7 4 1 7	H

Legenda: F = Page Fault

H = Hit (página encontrada)

- = Quadro livre

RESUMO DOS RESULTADOS



Algoritmo	Page Faults	Escritas em Disco	Taxa de Falhas (%)
FIFO	8	3	80,0%
LRU	8	3	80,0%
Random	8	3	80,0%

OBSERVAÇÕES



- No trace utilizado, os três algoritmos apresentaram o mesmo número de Page Faults.
- Isso ocorre porque o padrão de acesso é repetitivo e o conjunto de páginas é pequeno em relação à quantidade de quadros disponíveis.
- Traços maiores e mais complexos podem gerar diferenças significativas entre as políticas.



INTERPRETAÇÃO

Com 4 quadros disponíveis, o sistema consegue manter todas as páginas do conjunto de trabalho após os primeiros carregamentos, resultando em hits nas referências subsequentes (exceto quando há substituições necessárias).



MÉTRICAS UTILIZADAS

- Page Faults:** número de falhas de página durante a execução.
- Escritas em Disco:** número de páginas sujas removidas da memória.
- Taxa de Falhas (%):** $(\text{Page Faults} / \text{Total de Acessos}) \times 100$

- ✓ O simulador desenvolvido permite comparar o desempenho das políticas FIFO, LRU e Random de forma controlada e reprodutível.
- ✓ As métricas coletadas (Page Faults, Escritas em Disco e Taxa de Falhas) fornecem uma visão clara do impacto de cada política.
- ✓ A estrutura implementada é modular e pode ser facilmente expandida para novas políticas ou métricas.
- ✓ A ferramenta contribui para a escolha da melhor política de substituição em dispositivos de borda, promovendo maior eficiência e economia de recursos.
- ✓ Os resultados obtidos validam a importância da simulação na análise de desempenho e na tomada de decisão em ambientes com recursos limitados.

